

Student Name: Tushar Panchal

En. No. : 21162101014

Branch: CBA

Batch: 71

Subject: ML (Machine Learning)

EXPERIMENT 6

Design following model for given dataset and find the best suitable model using confusion matrix, f-score, accuracy, recall and precision matrices.

-decision tree

- Decision Tree
- Logistic Regression
- KNN Write down your conclusion at the end of the notebook..

```
In [ ]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, recall_score, precision_score
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
```

Importing Dataset

```
In [ ]: data = pd.read_csv('drug200.csv')
data.head(10)
```

Out[]:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY
5	22	F	NORMAL	HIGH	8.607	drugX
6	49	F	NORMAL	HIGH	16.275	drugY
7	41	M	LOW	HIGH	11.037	drugC
8	60	M	NORMAL	HIGH	15.171	drugY
9	43	M	LOW	NORMAL	19.368	drugY

Understanding the data

In []: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype  
---  -
 0   Age             200 non-null   int64  
 1   Sex             200 non-null   object  
 2   BP              200 non-null   object  
 3   Cholesterol      200 non-null   object  
 4   Na_to_K         200 non-null   float64 
 5   Drug            200 non-null   object  
dtypes: float64(1), int64(1), object(4)
memory usage: 9.5+ KB
```

In []: `data.describe()`

Out[]:

	Age	Na_to_K
count	200.000000	200.000000
mean	44.315000	16.084485
std	16.544315	7.223956
min	15.000000	6.269000
25%	31.000000	10.445500
50%	45.000000	13.936500
75%	58.000000	19.380000
max	74.000000	38.247000

In []: `data.isnull().sum()`

```
Out[ ]:      0
      Age  0
      Sex  0
      BP   0
      Cholesterol  0
      Na_to_K  0
      Drug  0
```

dtype: int64

Data Preprocessing

```
In [ ]: data.Sex.unique()
```

```
Out[ ]: array(['F', 'M'], dtype=object)
```

```
In [ ]: data.BP.unique()
```

```
Out[ ]: array(['HIGH', 'LOW', 'NORMAL'], dtype=object)
```

```
In [ ]: data.Cholesterol.unique()
```

```
Out[ ]: array(['HIGH', 'NORMAL'], dtype=object)
```

```
In [ ]: data.Drug.unique()
```

```
Out[ ]: array(['drugY', 'drugC', 'drugX', 'drugA', 'drugB'], dtype=object)
```

```
In [ ]: #Encoding categorical fields
```

```
le = LabelEncoder()
```

```
data['Sex'] = le.fit_transform(data['Sex'])
```

```
data['BP'] = le.fit_transform(data['BP'])
```

```
data['Cholesterol'] = le.fit_transform(data['Cholesterol'])
```

```
data['Drug'] = le.fit_transform(data['Drug'])
```

```
In [ ]: data.head(10)
```

```
Out[ ]:
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	0	0	0	25.355	4
1	47	1	1	0	13.093	2
2	47	1	1	0	10.114	2
3	28	0	2	0	7.798	3
4	61	0	1	0	18.043	4
5	22	0	2	0	8.607	3
6	49	0	2	0	16.275	4
7	41	1	1	0	11.037	2
8	60	1	2	0	15.171	4
9	43	1	1	1	19.368	4

```
In [ ]: x = data.drop('Drug' , axis = 1)
        y = data['Drug']
```

```
In [ ]: scaler = StandardScaler()

        x = scaler.fit_transform(x)
```

```
In [ ]: x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

        print("x train:",x_train.shape)
        print("x test:",x_test.shape)
        print("y train:",y_train.shape)
        print("y test:",y_test.shape)

x train: (160, 5)
x test: (40, 5)
y train: (160,)
y test: (40,)
```

Logistic Regression

```
In [ ]: log_model = LogisticRegression()

        log_model.fit(x_train , y_train)
```

```
Out[ ]: LogisticRegression ⓘ ?

LogisticRegression()
```

```
In [ ]: log_preds = log_model.predict(x_test)

        pd.DataFrame({'Actual':y_test , 'Predicted':log_preds})
```

Out[]:

	Actual	Predicted
86	3	3
120	4	4
22	4	4
11	4	4
195	2	2
2	2	2
121	4	4
94	4	4
66	0	0
63	3	3
108	1	1
96	4	4
138	1	0
65	4	4
188	4	4
155	2	2
24	4	4
99	4	4
153	3	4
46	0	0
178	4	4
139	3	3
143	4	1
74	4	4
186	1	1
169	0	0
101	0	0
197	3	3
109	4	4
177	4	4
57	4	4
106	3	3
160	3	3
84	2	2
124	1	1
85	1	1

	Actual	Predicted
126	4	4
183	4	4
80	1	1
116	3	3

```
In [ ]: log_acc = accuracy_score(y_test , log_preds)
log_prec = precision_score(y_test , log_preds , average = 'weighted')
log_recall = recall_score(y_test , log_preds , average = 'weighted')
log_f1 = f1_score(y_test , log_preds , average = 'weighted')

print("Accuracy:",log_acc)
print("Precision:",log_prec)
print("Recall:",log_recall)
print("F1 Score:",log_f1)
```

Accuracy: 0.925
Precision: 0.93
Recall: 0.925
F1 Score: 0.9255555555555557

```
In [ ]: log_cm = confusion_matrix(y_test , log_preds)
print(log_cm)
```

```
[[ 4  0  0  0  0]
 [ 1  5  0  0  0]
 [ 0  0  4  0  0]
 [ 0  0  0  7  1]
 [ 0  1  0  0 17]]
```

Decision Tree

```
In [ ]: dt_model = DecisionTreeClassifier()

dt_model.fit(x_train , y_train)
```

```
Out[ ]: ▼ DecisionTreeClassifier ⓘ ?
DecisionTreeClassifier()
```

```
In [ ]: dt_preds = dt_model.predict(x_test)

pd.DataFrame({'Actual':y_test , 'Predicted':dt_preds})
```

Out[]:

	Actual	Predicted
86	3	3
120	4	4
22	4	4
11	4	4
195	2	2
2	2	2
121	4	4
94	4	4
66	0	0
63	3	3
108	1	1
96	4	4
138	1	0
65	4	4
188	4	4
155	2	2
24	4	4
99	4	4
153	3	4
46	0	0
178	4	4
139	3	3
143	4	4
74	4	4
186	1	1
169	0	0
101	0	0
197	3	3
109	4	4
177	4	4
57	4	4
106	3	3
160	3	3
84	2	2
124	1	1
85	1	1

	Actual	Predicted
126	4	4
183	4	4
80	1	1
116	3	3

```
In [ ]: dt_acc = accuracy_score(y_test , dt_preds)
dt_prec = precision_score(y_test , dt_preds , average = 'weighted')
dt_recall = recall_score(y_test , dt_preds , average = 'weighted')
dt_f1 = f1_score(y_test , dt_preds , average = 'weighted')

print("Accuracy:",dt_acc)
print("Precision:",dt_prec)
print("Recall:",dt_recall)
print("F1 Score:",dt_f1)
```

Accuracy: 0.95
Precision: 0.9563157894736841
Recall: 0.95
F1 Score: 0.9497570297570299

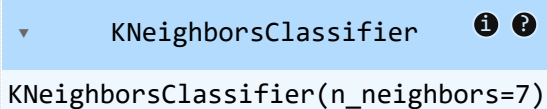
KNN

```
In [ ]: import math

math.sqrt(len(data))
```

Out[]: 14.142135623730951

```
In [ ]: knn = KNeighborsClassifier(n_neighbors=7)
knn.fit(x_train,y_train)
```

Out[]:  KNeighborsClassifier (i ?)
KNeighborsClassifier(n_neighbors=7)

```
In [ ]: knn_preds = knn.predict(x_test)

pd.DataFrame({'Actual':y_test , 'Predicted':knn_preds})
```


Out[]:

	Actual	Predicted
86	3	3
120	4	4
22	4	4
11	4	4
195	2	1
2	2	2
121	4	0
94	4	4
66	0	0
63	3	3
108	1	0
96	4	4
138	1	0
65	4	4
188	4	4
155	2	2
24	4	4
99	4	4
153	3	1
46	0	0
178	4	4
139	3	3
143	4	4
74	4	0
186	1	1
169	0	0
101	0	0
197	3	3
109	4	4
177	4	4
57	4	4
106	3	2
160	3	3
84	2	2
124	1	1
85	1	0

	Actual	Predicted
126	4	4
183	4	4
80	1	1
116	3	3

```
In [ ]: knn_acc = accuracy_score(y_test , knn_preds)
knn_prec = precision_score(y_test , knn_preds , average = 'weighted')
knn_recall = recall_score(y_test , knn_preds , average = 'weighted')
knn_f1 = f1_score(y_test , knn_preds , average = 'weighted')

print("Accuracy:",knn_acc)
print("Precision:",knn_prec)
print("Recall:",knn_recall)
print("F1 Score:",knn_f1)
```

```
Accuracy: 0.8
Precision: 0.8594444444444445
Recall: 0.8
F1 Score: 0.8133146265499207
```

```
In [ ]: knn_cm = confusion_matrix(y_test , knn_preds)
print(knn_cm)
```

```
[[ 4  0  0  0  0]
 [ 3  3  0  0  0]
 [ 0  1  3  0  0]
 [ 0  1  1  6  0]
 [ 2  0  0  0 16]]
```